Any given test double will have both an origin and a usage mode. For instance, you may have a pure double acting as a stub, or a verifying double acting as a spy.

We're going to explore the different kinds of doubles interactively in a live IRB session. Typically, mock object frameworks assume they're running inside an individual test, but RSpec's test doubles support a special stand-alone mode for these kinds of experiments.

To use this mode, fire up IRB and `require` the following file:

```
>> require 'rspec/mocks/standalone'
=> true
```

Keep this session running as you try the upcoming examples.

Usage Modes: Mocks, Stubs, and Spies

First, let's talk about the different usage modes of test doubles.

Generic Test Doubles

RSpec's `double` method creates a *generic test double* that you can use in any mode. The simplest way to call this method is with no arguments. Let's try that now:

```
>> ledger = double
=> #<Double (anonymous)>
```

In some ways, this double acts like an ordinary Ruby object. As you send messages to it (in other words, call methods on it), it will accept some messages and reject others.

The difference is that a generic double gives you more debugging information than a regular Ruby object. Try sending the `record` message to your test double now:

```
>> ledger.record(an: :expense)
RSpec::Mocks::MockExpectationError: #<Double (anonymous)> received ↩
unexpected message :record with ({:an=>:expense})
    « backtrace truncated »
```

When we sent this message, the double raised an exception. Doubles are *strict* by default: they will reject all messages except the ones you've specifically allowed. We'll see how to do that later on.